



On the Minimization of Graph Counterfactual Explanations: Theory and a Local Bounded Search Algorithm

Rodrigo García¹ · Mario Alfonso Prado-Romero² · Francesco Gullo³ · Giovanni Stilo⁴

Received: 28 February 2026 / Revised: 19 July 2026 / Accepted: 25 August 2026
© The Author(s) 2026

Abstract

Graph counterfactual explainability (GCE) addresses the interpretability limitations of opaque machine learning models on graph-structured data by producing graph counterfactuals (GCs): alternative graphs that remain maximally similar to a given instance while inducing a different model prediction. State-of-the-art GCE methods usually follow a generate-and-minimize pipeline: first, generate a valid counterfactual, then refine it to be closer to the original graph. While generation is well-studied, minimization still lacks a formal definition, complexity analysis, and general-purpose algorithms; existing solutions are either simple random edge-swap heuristics or tightly coupled to specific generators, limiting effectiveness and generality. In this work, we address a key gap in GCE by providing the first principled study of minimizing a given valid graph counterfactual. We formalize the task as an optimization problem and prove it is NP-hard. We then propose a decoupled generate-and-minimize framework and introduce Local Bounded Search (LBS), a model-agnostic heuristic that refines any valid counterfactual via constrained structural and attribute edits to reduce dissimilarity while preserving validity. Across nine synthetic and real-world datasets (molecular, biomedical, social), LBS reduces structural edit distance by up to 98% from the initial counterfactual and consistently outperforms existing refinement heuristics.

Keywords Machine learning · Graphs · Explainable AI · Counterfactual explainability · Local search

1 Introduction

Modern machine learning models, including deep neural networks, transformers, and graph neural networks, have achieved remarkable performance across a wide range of *artificial intelligence* (AI) tasks. However, their inherent *black-box* nature Petch et al. (2021) limits the *interpretability* of their outcomes, a critical issue in sensitive domains such as health-care and finance, where *explainability* is essential for trustworthy decisions Guidotti et al.

Editors: Gianvito Pio, Jurica Levatić, Nikola Simidjievski.

Extended author information available on the last page of the article

(2018). The field of *eXplainable AI* (XAI) Miller (2019) addresses this need by designing *post-hoc explainability* methods that provide human-readable *explanations* of how a black-box model works and why it produces specific outcomes.

Among the various forms of post-hoc explanation Bodria et al. (2023), a *counterfactual explanation* (or, simply, *counterfactual*) of an instance $\vec{x}$ w.r.t. a model Φ is another instance $\vec{x}'$ as close as possible to $\vec{x}$ such that $\Phi(\vec{x}') \neq \Phi(\vec{x})$ (Karimi et al., 2023; Wachter et al., 2018). Counterfactuals provide actionable recourse for individuals affected by model decisions Jiang et al. (2024); e.g., “If your income had been £45,000, you would have been offered a loan” Wachter et al. (2018).

Graphs, collections of entities (*nodes*) connected through relationships (*edges*), are a ubiquitous model for representing relational data across domains such as chemistry, biology, and social networks (Aggarwal & Wang, 2010; Frieze et al., 2013). *Graph machine learning* (GML) Ma and Tang (2021) applies machine learning to graph-structured data, with tasks including *graph classification* (e.g., diagnosing disease from brain connectivity networks Abrate and Bonchi (2021) or predicting chemical properties of molecular graphs (Wellawatte et al., 2022; Wu et al., 2018)) and *graph regression*, e.g., predicting solubility or quantum mechanical properties of compounds (Wu et al., 2018; Zhang & Zhao, 2021).

Graph counterfactual explainability (GCE) contextualizes counterfactual reasoning to graph-structured data Prado-Romero et al. (2024). A *graph counterfactual* (GC) of a graph G w.r.t. a model Φ is a graph G' as close as possible to G in topology and node attributes, such that $\Phi(G') \neq \Phi(G)$. Ideally, a GC should differ from G by a minimal set of graph edits, such as edge additions/removals or node-attribute modifications.

1.1 Motivation

Most state-of-the-art GCE methods embrace fully or partially (only generation) the *generate-and-minimize* paradigm (Abrate & Bonchi, 2021; Wellawatte et al., 2022; Prado-Romero et al., 2024; Faber et al., 2020; Liu et al., 2021; Prenkaj et al., 2025): a *valid GC* G' with $\Phi(G') \neq \Phi(G)$ is first generated and then refined to minimize its difference from G while preserving validity (Figure 1). Both steps are important: generation ensures correctness, while minimization ensures that the GC is close to the original, producing concise,

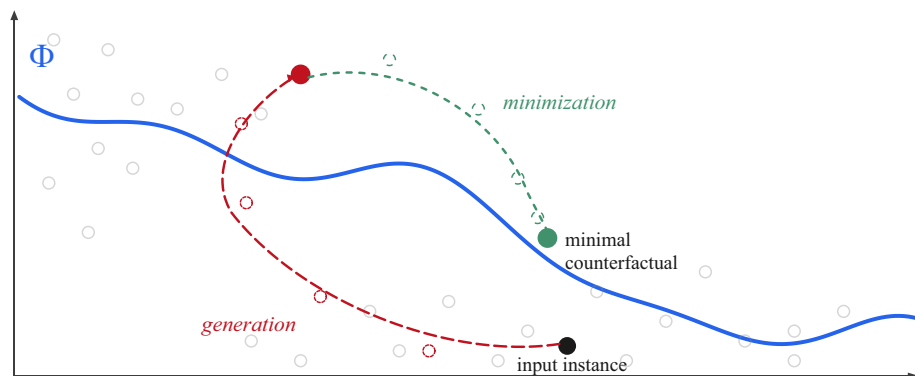


Fig. 1 A generator first produces a valid counterfactual (red) that crosses the model’s decision boundary. A minimizer then refines it into a minimal counterfactual (green) closer to the original graph. This work focuses on the often-overlooked minimization step in generate-and-minimize GCE

sparse explanations that are easier to inspect (proximity is a prerequisite for semantic plausibility, but by itself does not guarantee it). Despite this, the literature focuses primarily on generation, while existing minimization approaches either follow a simple random edge-swap strategy Abrate and Bonchi (2021) or are tightly coupled with the generation phase (Wellawatte et al., 2022; Faber et al., 2020; Liu et al., 2021; Prenkaj et al., 2025), limiting the capability of optimizing minimization independently.

1.2 Contributions

In this work, we fill this gap by providing the first explicit and principled investigation of GC minimization. We formalize it as the GC-MIN optimization problem and prove its NP-hardness. To tackle GC-MIN, we devise Local Bounded Search (LBS), a novel local-search algorithm that refines any valid counterfactual by exploring bounded neighborhoods of graph edits, reducing dissimilarity to the original graph while preserving validity in a fully black-box setting. By decoupling minimization from generation, our framework also enables systematic and fair comparison of different counterfactual generators.

1.3 Summary and Roadmap

In this work, we achieve the following main contributions:

- We provide the first theoretical and empirical in-depth investigation of the minimization phase of the generate-and-minimize category of GCE approach. We formalize such a minimization phase by introducing the GC-MIN optimization problem.
- We show that the proposed GC-MIN problem is NP-hard.
- To tackle GC-MIN, we devise a novel Local Bounded Search (LBS) algorithm that refines any given valid counterfactual by exploring constrained neighborhoods so as to minimize graph edits while preserving validity.
- We extensively evaluate the proposed LBS algorithm across a variety of synthetic and real-world datasets from heterogeneous domains. Results attest that LBS consistently reduces GC size over baseline minimizers.

The remainder of this paper is organized as follows: Section 2 reviews related work; Section 3 introduces and theoretically characterizes GC-MIN; Section 4 presents LBS; Section 5 reports the experimental evaluation; Section 6 concludes with future directions.

2 Related Work

Prado-Romero et al. (2024) classify GCE methods into three main categories, namely: *search-based methods*, *heuristic-based methods*, and *learning-based methods*.

Search-based methods explore discrete search spaces to identify valid counterfactual graphs. Prominent examples include *GNN-MOExp* Liu et al. (2021) and *DCE* Faber et al. (2020). These approaches produce reliable explanations within their predefined search space, but do not guarantee minimality w.r.t. the full combinatorial space of graph edits.

Heuristic-based methods rely on heuristic perturbation strategies that alter characteristics of the input graph until a change in model prediction is obtained. Methods within this category include *MACCS* Wellawatte et al. (2022), *obs*, and *DDBS* Abrate and Bonchi (2021).

Learning-based methods learn perturbation mechanisms or latent representations to directly produce GCs, including *CF²* Tan et al. (2022), *CLEAR* Ma et al. (2022), *RSGG-CE* Prado-Romero et al. (2024), and *GIST* Prenkaj et al. (2025). Notably, *GIST* introduces a backtracking process that overshoots the decision boundary and then refines the counterfactual, sharing the conceptual motivation of a generate-and-minimize framework but remaining integrated within a single model.

All the above methods (except *obs* and *DDBS*) primarily address GC generation, enforcing minimality only implicitly through regularization or heuristic constraints. *obs* and *DDBS* do include an explicit minimization phase that could in principle be separated from generation, but it consists of a simple random edge-swapping strategy lacking formal grounding. We use it as a baseline in our experiments.

From a broader optimization perspective, our approach draws on the rich literature of iterative local search for NP-hard combinatorial problems (Hoos & Stützle, 2004; Glover & Kochenberger, 2010; Michiels et al., 2007). Key design elements of LBS, namely first-improvement acceptance, bounded neighborhoods, and overshooting to escape local minima, are rooted in established local search principles. However, the application to graph counterfactual minimization introduces domain-specific challenges, including the interplay between structural and attribute edits, the need to preserve oracle validity at every step, and the black-box nature of the feasibility constraint.

To the best of our knowledge, no prior GCE method isolates minimization as an independent optimization problem. In this work, we decouple minimization from generation, enabling fair comparison of refinement strategies across heterogeneous generators and a principled focus on the minimization phase.

3 Problem Definition

We deal with data represented as *node-attributed graphs*, i.e., graphs of the form $G = (V, E, X)$, where V is a set of vertices, $E \subseteq V \times V$ is a set of edges, and $X \in \mathbb{R}^{|V| \times d}$ is a matrix where the i -th row corresponds to a d -dimensional real-valued vector of attributes for node i , for $i = 1, \dots, |V|$. The graphs we manage can be either undirected or directed: our problem statement, definitions, theoretical properties, and algorithms work for both cases. Similarly, we can also smoothly handle the special case of graphs with no node attributes, i.e., corresponding to the case $d = 0$. For the sake of simplicity, we hereinafter term a node-attributed graph simply “graph”.

We focus on machine learning tasks performed on (node-attributed) graphs. In particular, we assume the availability of a machine learning model, formally represented as a function $\Phi : \mathcal{G} \rightarrow \mathbb{R}$ that assigns a real value to any graph G belonging to the set $\mathcal{G}$ of all the possible graphs. In our context, we treat Φ as a black box: we assume to know nothing about its internals (i.e., about how it was trained or performs inference). Also, we do not restrict Φ to be peculiar of any specific machine learning task: depending on the output, it can represent any task, e.g., classification or regression. Due to its black-box nature, we alternatively and equivalently refer to the available machine learning model Φ as *oracle*.

Given a graph $G = (V, E, X) \in \mathcal{G}$, a *graph edit* (or, simply, an *edit*) in G is a modification of G in terms of topology (i.e., edges) or attributes. Specifically, a graph edit may be of three kinds: (i) addition of an edge $e \notin E$, (ii) removal of an edge $e \in E$, (iii) change of a node attribute X_{ij} into $X'_{ij} \neq X_{ij}$. A *graph counterfactual* (GC) $G' = (V, E', X')$ of a graph $G = (V, E, X)$ is a graph obtained by performing one or more edits to G , i.e., such that either $E' \neq E$ or $X' \neq X$ (or both), and such that the output $\Phi(G')$ of model Φ on G' is other than the output $\Phi(G)$ of Φ on G , i.e., $\Phi(G') \neq \Phi(G)$. We denote by $\Delta(G, G')$ the set of edits that intervene between graphs G and G' . Note that for G' to be a GC of G , both $\Phi(G') \neq \Phi(G)$ and $\Delta(G, G') \neq \emptyset$ must in principle hold. However, $\Delta(G, G') \neq \emptyset$ is clearly implied by $\Phi(G') \neq \Phi(G)$. Thus, stating just the latter suffices to indicate that G' is a GC of G .

We introduce the GC-MIN problem, which consists of modifying a given GC G' of a graph G into G'' so that G'' is still a GC and the number of edits intervening between G and G'' is minimal (i.e. $G'' = G^*$ and $G'' \neq G'$). Formally:

Problem 1 (GC-MIN) Given a machine learning model (oracle) $\Phi : \mathcal{G} \rightarrow \mathbb{R}$, a graph $G = (V, E, X) \in \mathcal{G}$, and a GC $G' = (V, E', X') \in \mathcal{G}$ of G , modify G' so as to find G'' as

$$G^* = \operatorname{argmin}_{G'' \in \mathcal{G} | \Phi(G'') \neq \Phi(G)} |\Delta(G, G'')|. \quad (1)$$

Note that the GC-MIN problem is well-defined even if no starting GC G' is provided as part of the input. This means that the ultimately desired G^* could in principle be obtained even from scratch. However, we do require a GC G' as input because we would like to emphasize the fact that the tackled problem should actually search for G^* *starting from* G' . This is required to faithfully model the minimization step of the aforementioned generate-and-minimize approach to GCE. Notations used in this work are reported in Table 1.

Theoretical characterization of Problem 1. In the following, we show fully general hardness results for our GC-MIN problem. Specifically, we prove that GC-MIN is NP-hard regardless of the underlying machine learning task or machine learning model. We also demonstrate that its NP-hardness holds for both general inputs (i.e., for the most general case where the input graph and GC are directed) and for the special case of input undirected graphs. Our results complement prior theoretical work about intractability of (G)CE prob-

Table 1 Main notations used in this work

Notation	Description
$G = (V, E, X)$	Input (attributed) graph
G'	Graph counterfactual (GC) of G of the generation phase
G''	Graph counterfactual (GC) of G of the minimization phase
G^*	Minimal GC of G
$\Phi : \mathcal{G} \rightarrow \mathbb{R}$	Machine learning model (oracle)
$\Delta(G, G')$	Set of graph edits to transform G into G'
$E(V) = \binom{V}{2}$	Set of all possible structural edits
$S \subseteq E(V)$	Solution: structural edit set ($G'' = G \oplus S$)
$\hat{S}$	Best valid solution found so far
$b = \hat{S} $	Cardinality bound on solutions
$\mathcal{N}(S)$	Neighborhood of solution S
$\mathcal{A} = \langle \alpha_1, \dots, \alpha_m \rangle$	Node-attribute modification strategies

lems under specific (i.e., non model-agnostic or machine-learning-task-dependent) conditions (Dimitriou et al., 2024; Qiu et al., 2024; Verma et al., 2023). In deriving our results, we assume that the output $\Phi(G)$ of applying the given machine learning model (oracle) Φ to a graph G is computable in polynomial time in the size of G . This is a reasonable assumption: in fact, if this does not hold, then the inference phase according to Φ would be not polynomial and thus useless in many practical contexts.

Theorem 1 (*NP-hardness of Problem 1 for general graphs*) Assuming $\Phi(G)$ to be computable in polynomial time in the size of G , Problem 1 executed on an oracle Φ , a graph $G = (V, E, X)$, and a GC $G' = (V, E', X')$ of G is NP-hard.

Proof We prove NP-hardness of the target optimization problem by showing NP-completeness of the corresponding decision version of the problem, which can be stated as follows: Given Φ , G , G' , and an integer $k \geq 0$, is there any set $\Delta(G, G'')$ of edits to transform G into G'' such that $|\Delta(G, G'')| \leq k$ and $\Phi(G'') \neq \Phi(G)$? Such a decision problem is clearly in NP, as any candidate set $\Delta(G, G'')$ of edits can be verified to be a yes-instance or not on input $\langle \Phi, G, G', k \rangle$ in polynomial time in the size of the input. In fact, one can check whether $|\Delta(G, G'')| \leq k$ in constant time, compute G'' in $O(|V|^2 + |X|)$ time (as $|\Delta(G, G'')|$ is at most $|V|^2 + |X|$), and, by hypothesis, compute $\Phi(G)$ and $\Phi(G'')$ in time polynomial in the size of G and G'' (which is at most $|V|^2 + |X|$), respectively.

As for the NP-hardness, we show a reduction from the well-established FEEDBACK ARC SET (FAS) problem, which is one of Karp's classic NP-complete problems Karp (1972). The (decision version of the) FAS problem is as follows: Given a directed graph $D = (N, A)$ and an integer $h > 0$, is there any subset $R \subseteq A$ with $|R| \leq h$ whose removal from A renders D acyclic? Given an instance $I = \langle D = (N, A), h \rangle$ of FAS, we construct an instance $I' = \langle \Phi, G = (V, E, X), G', k \rangle$ of GC-MIN by setting $G = G' = D$, $k = h$, and Φ to a function that returns 1 if the graph given as input is acyclic, 0 otherwise. Clearly, I' can be constructed in polynomial time, and the $\Phi(G)$ takes polynomial time in the size of G , as checking whether a graph is acyclic is a polynomial-time problem. We distinguish two cases, namely (1) D is acyclic, and (2) D is not acyclic. As for Case 1, clearly, the given FAS instance I is always a yes-instance, regardless of h . In terms of the corresponding GC-MIN instance I' , it holds that $\Phi(G) = 1$ (as $G = D$ is acyclic), and a GC G'' can be easily constructed by just adding a self-loop to G . This way, $\Phi(G'') = 0$ (as G'' contains now a cycle due to the added self-loop) and $|\Delta(G, G'')| = 1 \leq k$ (as $k > 0$). This makes I' a yes-instance for GC-MIN. As for Case 2, I could be either a yes- or a no-instance for FAS. If I is a yes-instance, then there exists a subset $R \subseteq A$ of arcs of size $|R| \leq h$ whose removal makes D acyclic. Thus, the graph defined as $G'' = (V, E \setminus R, X)$ would be a GC of G (as $\Phi(G) = 0 \neq \Phi(G'') = 1$), and $|\Delta(G, G'')| = |R| \leq h = k$, which makes I' a yes-instance for GC-MIN. Instead, if I is a no-instance for FAS, then there exists no subset of arcs of size $\leq h$ whose removal makes D acyclic. Translated to the I' instance, this means that there exists no set of edits of size $\leq h = k$ to construct a GC of G . Thus, I' is a no-instance for GC-MIN. $\square$

Theorem 2 (*NP-hardness of Problem 1 for undirected graphs*) Assuming $\Phi(G)$ to be computable in polynomial time in the size of G , Problem 1 executed on an oracle Φ , an undirected graph $G = (V, E, X)$, and a GC $G' = (V, E', X')$ of G is NP-hard.

Proof sketch Similarly to Theorem 1, we prove NP-hardness by showing NP-completeness of the decision version of GC-MIN (stated in the proof of Theorem 1. Membership in NP can be easily checked to hold with arguments analogous to the ones reported in the proof of Theorem 1. As for NP-hardness, here we reduce from the EDGE BIPARTIZATION problem (also known as BIPARTITE EDGE DELETION), which is known to be NP-complete Garey and Johnson (1990): given an undirected graph, decide whether it is possible to delete at most h edges to make it bipartite. Given an instance $I = \langle F, h \rangle$ of EDGE BIPARTIZATION, we construct an instance $I' = \langle \Phi, G, G', k \rangle$ of GC-MIN by setting $G = G' = F$, $k = h$, and Φ to a function that returns 1 if the graph given as input is bipartite, 0 otherwise. At this point, NP-hardness can be proved in a way very similar to Theorem 1, just by replacing the concept of acyclicity with the concept of bipartiteness. $\square$

4 Local Bounded Search

The hardness results of the previous section motivate approximate algorithms for GC-MIN. We devise Local Bounded Search (LBS), an iterative local-search algorithm that takes an oracle Φ , a graph $G = (V, E, X)$, and a GC $G' = (V, E', X')$ of G , and seeks to minimize G' into G'' such that $|\Delta(G, G'')| \leq |\Delta(G, G')|$ while ensuring $\Phi(G'') \neq \Phi(G)$. We first introduce the edit operations of LBS: structural edits (edge addition/removal) and the node-attribute modification strategies that accompany them (Section 4.1). We then present the search procedure of LBS (Section 4.2).

A *structural edit* of G is the removal of an existing edge or the addition of an absent one; the set of all possible structural edits is $E(V) = \binom{V}{2}$ (node-attribute modifications are handled in Section 4.1). A *solution* $S \subseteq E(V)$ represents the edits that transform G into $G'' = G \oplus S$ (i.e., $S \equiv \Delta(G, G'')$). We denote by $\mathcal{P}(E(V))^+$ the set of valid GCs. The initial solution $S_0 = \Delta(G, G')$ corresponds to the input counterfactual; S_i denotes the solution at iteration i . To explore the search space $\mathcal{P}(E(V))^+$, LBS defines three operations: removal $\text{del}(S_i, e) = S_i \setminus \{e\}$, addition $\text{add}(S_i, e) = S_i \cup \{e\}$ ($e \notin S_i$), and replacement $\text{swap}(S_i, e, e') = S_i \setminus \{e\} \cup \{e'\}$ ($e \in S_i, e' \notin S_i$).

4.1 Node-Attribute Modification Strategies

Because the oracle Φ uses the full attributed graph, each structural candidate is paired with a sequence of attribute-modification strategies $\mathcal{A} = \langle \alpha_1, \dots, \alpha_m \rangle$ (with $m = |\mathcal{A}| = 8$). The eight strategies we propose span: (i) no-change, (ii) increasingly expressive local neighborhood aggregations, and (iii) global diffusion operators (e.g., Laplacian/heat-kernel/random-walk), providing diverse attribute hypotheses without domain tuning. Strategies are tried in a cheapest-to-richest order under a first-accept policy, so ordering affects efficiency (oracle calls) but not the set of reachable solutions. These are optimization operators for finding attribute configurations that preserve validity under the oracle, not claims of semantically interpretable edits. In domains with strict semantic constraints (for example, the chemical validity of molecules) the set of operators can be restricted or combined with a domain validity check. Let $F = X(G_{\text{struct}})$ denote the node-feature matrix of G_{struct} (i.e., the attri-

butes inherited from G after applying the structural edits), A its adjacency matrix, D the diagonal degree matrix, and $N(i)$ the set of neighbours of node i in G_{struct} .

4.1.1 No Changes

The node attributes are left unmodified: $X'' = F$. This strategy is appropriate when node attributes are exogenous to the graph topology and serves as a low-cost baseline.

4.1.2 Average Smoothing

Each node's features are replaced by the mean of its neighbours' features: $f'_i = \frac{1}{|N(i)|} \sum_{j \in N(i)} f_j$

. This promotes local homogeneity and smoother signal propagation Shuman et al. (2013). Features of disconnected nodes remain unchanged.

4.1.3 Average Smoothing with Disconnection

Analogous to average smoothing, but nodes that become disconnected after perturbation have their attributes set to the null vector: $f'_i = 0$ if $N(i) = \emptyset$.

4.1.4 Weighted Smoothing

Each neighbour's contribution is weighted by its degree: $f'_i = \sum_{j \in N(i)} w_{ij} f_j$, where

$w_{ij} = d_j / \sum_{k \in N(i)} d_k$ and d_j is the degree of node j Sandryhaila and Moura (2013).

4.1.5 Graph Laplacian Regularisation

The graph Laplacian $L = D - A$ smooths features via $F' = (1 - \lambda_L)F - \lambda_L LF$, where λ_L controls the smoothness strength (Belkin & Niyogi, 2003; Zhou et al., 2004).

4.1.6 Feature Aggregation from New Neighbours

A convex combination of original features and the neighbourhood mean: $f'_i = \alpha f_i + (1 - \alpha) \frac{1}{|N(i)|} \sum_{j \in N(i)} f_j$, where $\alpha \in [0, 1]$ controls retention of original features (Kipf & Welling, 2016; Hamilton et al., 2017).

4.1.7 Heat-kernel Diffusion

Feature propagation is modelled as a heat-diffusion process: $F' = \exp(-tL)F$, where $t > 0$ controls the diffusion extent Coifman and Lafon (2006).

4.1.8 Random-Walk Diffusion

Features are updated via a k -step random walk with transition matrix $P = D^{-1}A$: $F' = P^k F$, where k is the number of steps Kondor and Borgwardt (2008).

4.2 Search Procedure

The neighborhood $\mathcal{N}(S_i)$ of S_i is the set of solutions reachable by a single edit operation; its size depends combinatorially on $|S_i|$ and $|E(V)|$ and shrinks as $|S_i|$ decreases.

4.2.1 Cardinality-Bounded Exploration

Minimizing $|\Delta(G, G'')|$ (Eq. 1) is equivalent to minimizing $|S|$. Once a valid solution with cardinality $b = |S_i|$ is found, solutions exceeding b cannot improve the objective. LBS therefore enforces a cardinality bound: *deletions* are prioritized as they directly reduce $|S|$; *replacements* preserve cardinality while enabling further reductions; *additions* are permitted only as compensating moves when $|S| < b$, allowing limited backtracking without exceeding the current bound.

The bounding is part of the design and never sacrifices reachability, since LBS explores solutions that need not be valid and a replacement can substitute several edits: any valid counterfactual with cardinality smaller than b can be reached without having to explore solutions of sizes that exceed b .

Proposition 1 (*Reachability under the cardinality bound*) *Let $\hat{S}$ be a valid solution with $|\hat{S}| = b$, and let S^* be any valid solution with $|S^*| \leq b$. There exists a sequence of LBS operations from $\hat{S}$ to S^* that never visits a solution of cardinality greater than b . Consequently, restricting the search to solutions of size at most b never excludes an optimum, and LBS never needs to exceed b .*

The limitation caused by the bounding is, therefore, purely probabilistic. Reaching the optimal solution depends on exploring the correct solutions within the budget of oracle calls. The overshooting (Strategy 1b) and replacement (Strategy 2) strategies seek to increase this probability.

4.2.2 Priority-Ordered First-Improvement Exploration

LBS adopts a *first-improvement* acceptance policy (Hoos & Stützle, 2004; Glover & Kochenberger, 2010; Michiels et al., 2007): candidates are evaluated sequentially and the first strict improvement is accepted, substantially reducing per-iteration cost while preserving monotonic progress. Under this policy, the algorithm's behaviour is governed by the order in which strategies are attempted, yielding a *priority-ordered exploration* scheme.

Let $\hat{S}$ denote the best valid solution found so far, with bound $b = |\hat{S}|$. The exploration strategies are *mutually exclusive*: at every iteration exactly one is executed, always operating on $\hat{S}$. Strategies are attempted in fixed priority order; the first to succeed concludes the iteration. If a strategy fails, LBS falls through to the next, restarting from $\hat{S}$.

Since Φ operates on the full graph $G'' = (V, E'', X'')$, node attributes may need adjustment after structural edits. Each structural candidate S' is therefore evaluated under a sequence of *node-attribute modification strategies* $\mathcal{A} = \langle \alpha_1, \dots, \alpha_m \rangle$, each producing $X'' = \alpha_j(G \oplus S')$. The first variant satisfying $\Phi((V, E(G \oplus S'), X'')) \neq \Phi(G)$ is accepted; if none succeeds, the candidate is rejected. This is encapsulated in `EVALWITHATTRVARIANTS` (Algorithm 1); the strategies in $\mathcal{A}$ are detailed in Section 4.1.

The four strategies, listed from highest to lowest priority, are as follows.

- 1a. Single-edit removal (highest priority).** An edit $e \in \hat{S}$ is sampled uniformly at random and $S' = \hat{S} \setminus \{e\}$ is evaluated via `EVALWITHATTRVARIANTS`. Since $|S'| = b - 1$, every valid deletion strictly improves the objective. If valid, update $\hat{S} \leftarrow S'$, $b \leftarrow |S'|$, and proceed to the next iteration. Up to T_{del} attempts are made; if none succeeds, fall through to Strategy 1b.
- 1b. Shrinking-overshooting.** When single-edit removals fail, individual edits are likely *mutually supportive* (each necessary alone, yet subsets jointly redundant). To escape such plateaux, LBS removes $k > 1$ edits simultaneously. The removal size k is drawn from a truncated exponential distribution over $\{1, \dots, \lfloor b/2 \rfloor\}$, biased toward larger values:

$$p(k) = \frac{e^{-\lambda(\lfloor b/2 \rfloor - k)}}{\sum_{j=1}^{\lfloor b/2 \rfloor} e^{-\lambda(\lfloor b/2 \rfloor - j)}}, \quad k \in \{1, \dots, \lfloor b/2 \rfloor\}, \quad (2)$$

where $\lambda > 0$ is the rate parameter. A random subset $R \subseteq \hat{S}$ with $|R| = k$ is removed, and $S' = \hat{S} \setminus R$ is evaluated via `EVALWITHATTRVARIANTS`. If valid, $\hat{S}$ is updated but b is *not* reduced, so that compensating additions (Strategy 3) remain possible. Otherwise, fall through to Strategy 2.

- 2. Replacement (medium priority).** A pair (e, e') is sampled from $\hat{S} \times (E(V) \setminus \hat{S})$ and $S' = \hat{S} \setminus \{e\} \cup \{e'\}$ is evaluated via `EVALWITHATTRVARIANTS`. Cardinality is preserved ($|S'| = b$), but the restructured solution may unlock further deletions. Up to T_{swap} trials are made; if none succeeds, fall through to Strategy 3.
- 3. Compensating addition (lowest priority).** Attempted only if $|\hat{S}|$ has been reduced below a previously recorded bound in an earlier iteration. For each $e \in E(V) \setminus \hat{S}$, evaluate $S' = \text{add}(\hat{S}, e) = \hat{S} \cup \{e\}$ via `EVALWITHATTRVARIANTS`, subject to $|S'| < b$. This allows limited backtracking without ever exceeding the best-known bound.

If none of the four strategies produces an acceptable candidate, the algorithm terminates and returns $\hat{S}$. Algorithm 1 presents the `EVALWITHATTRVARIANTS` subroutine, and Algorithm 2 provides the complete pseudocode of LBS.

Input: Oracle Φ ; original graph $G = (V, E, X)$; structural candidate S' ; attribute strategies $\mathcal{A} = \langle \alpha_1, \dots, \alpha_m \rangle$ ($m = |\mathcal{A}|$)

Output: **true** and the valid attributed graph (S', X'') if a variant succeeds; **false** otherwise

```

1:  $G_{\text{struct}} \leftarrow G \oplus S'$  ▷ Apply structural edits
2: for  $j = 1, \dots, m$  do
3:    $X'' \leftarrow \alpha_j(G_{\text{struct}})$  ▷ Compute attribute variant
4:    $G'' \leftarrow (V, E(G_{\text{struct}}), X'')$ 
5:   if  $\Phi(G'') \neq \Phi(G)$  then
6:     return (true,  $S'$ ,  $X''$ )
7:   end if
8: end for
9: return (false,  $\perp$ ,  $\perp$ )

```

Input: Φ , $G = (V, E, X)$, valid GC $G' = (V, E', X')$, T_{del} , T_{swap} , λ , $\mathcal{A}$

Output: Minimized valid GC G'' with $|\Delta(G, G'')| \leq |\Delta(G, G')|$

Notation: $\text{EVAL}(S') \equiv \text{EVALWITHATTRVARIANTS}(\Phi, G, S', \mathcal{A})$

```

1:  $\hat{S} \leftarrow \Delta(G, G')$ ;  $\hat{X} \leftarrow X'$ ;  $b \leftarrow |\hat{S}|$ 
2: repeat
3:    $\text{acc} \leftarrow \text{false}$ 
Strategy 1a: Single-edit removal
4:   for  $t = 1, \dots, T_{\text{del}}$  do
5:     Sample  $e \in \hat{S}$  u.a.r.;  $(v, S', X'') \leftarrow \text{EVAL}(\hat{S} \setminus \{e\})$ 
6:     if  $v$  then  $\hat{S} \leftarrow S'$ ;  $\hat{X} \leftarrow X''$ ;  $b \leftarrow |S'|$ ;  $\text{acc} \leftarrow \text{true}$ ; break
7:     end if
8:   end for
Strategy 1b: Shrinking-overshooting
9:   if  $\neg \text{acc}$  and  $b \geq 2$  then
10:    Sample  $k \sim p(\cdot)$  (Eq. 2); sample  $R \subseteq \hat{S}$ ,  $|R|=k$  u.a.r.
11:     $(v, S', X'') \leftarrow \text{EVAL}(\hat{S} \setminus R)$ 
12:    if  $v$  then  $\hat{S} \leftarrow S'$ ;  $\hat{X} \leftarrow X''$ ;  $\text{acc} \leftarrow \text{true}$  ▷  $b$  not updated: allows compensating additions
13:    end if
14:   end if
Strategy 2: Replacement
15:   if  $\neg \text{acc}$  then
16:     for  $t = 1, \dots, T_{\text{swap}}$  do
17:       Sample  $e \in \hat{S}$ ,  $e' \in E(V) \setminus \hat{S}$  u.a.r.
18:        $(v, S', X'') \leftarrow \text{EVAL}((\hat{S} \setminus \{e\}) \cup \{e'\})$ 
19:       if  $v$  then  $\hat{S} \leftarrow S'$ ;  $\hat{X} \leftarrow X''$ ;  $\text{acc} \leftarrow \text{true}$ ; break
20:       end if
21:     end for
22:   end if
Strategy 3: Compensating addition
23:   if  $\neg \text{acc}$  and  $|\hat{S}|+1 < b$  then
24:     for  $e \in E(V) \setminus \hat{S}$  do
25:        $(v, S', X'') \leftarrow \text{EVAL}(\hat{S} \cup \{e\})$ 
26:       if  $v$  then  $\hat{S} \leftarrow S'$ ;  $\hat{X} \leftarrow X''$ ;  $b \leftarrow |S'|$ ;  $\text{acc} \leftarrow \text{true}$ ; break
27:       end if
28:     end for
29:   end if
30: until  $\neg \text{acc}$ 
31: return  $G'' = (V, E(G \oplus \hat{S}), \hat{X})$ 

```

4.2.3 Strategy Recurrence and Two-Stage Deletion Rationale

Since the priority ordering is re-evaluated from scratch at every iteration, the same strategy can be selected consecutively. In practice, the execution exhibits characteristic *phases*: Strategy 1a dominates while redundant edits exist, until no single removal is viable; Strategy 1b then aggressively restructures $\hat{S}$, after which Strategy 1a can regain control. Similarly, a successful replacement (Strategy 2) may re-enable single-edit removals.

The two-stage deletion design reflects complementary roles: Strategy 1a is a lightweight probe that succeeds when individual edits are redundant, while Strategy 1b escapes local minima where edits are *mutually supportive* (each necessary alone, but groups jointly replaceable) by removing k edits simultaneously. The exponential-decay sampling (Eq. 2) biases toward larger k , while retaining non-zero probability for smaller values to avoid excessive overshooting.

4.2.4 Correctness, Termination, and Complexity

At every iteration, exactly one strategy is executed. The accepted move either (i) strictly decreases $|\hat{S}|$ and updates b (Strategy 1a), (ii) strictly decreases $|\hat{S}|$ without updating b , enabling compensating additions (Strategy 1b), (iii) preserves cardinality while restructuring the solution (Strategy 2), or (iv) increases cardinality by one while remaining below b (Strategy 3). A candidate S' is accepted only if `EVALWITHATTRVARIANTS` finds at least one $\alpha_j \in \mathcal{A}$ such that $\Phi((V, E(G \oplus S'), \alpha_j(G \oplus S'))) \neq \Phi(G)$; hence $\hat{S}$ is updated exclusively with valid counterfactuals. Since b is non-increasing, $|\hat{S}| \leq b$, and $\mathcal{P}(E(V))^+$ is finite, the algorithm terminates in a finite number of iterations. Attribute variant selection multiplies oracle calls per candidate by at most $|\mathcal{A}|$ but does not alter the termination argument. Per iteration, Strategy 1a requires at most $T_{\text{del}} \cdot |\mathcal{A}|$ oracle calls, Strategy 1b at most $|\mathcal{A}|$, Strategy 2 at most $T_{\text{swap}} \cdot |\mathcal{A}|$, and Strategy 3 $\mathcal{O}(|E(V)| \cdot |\mathcal{A}|)$. Due to first-improvement acceptance and mutual exclusivity, only one strategy incurs cost per iteration, terminating as soon as a valid improving candidate is found. Finally, if the provided seed G' is not a valid counterfactual (that is, $\Phi(G') = \Phi(G)$), LBS detects it before refining and returns it unchanged. Guaranteeing a valid seed is the responsibility of the generator (Section 5).

4.2.5 When LBS Is Supposed to Work More or Less Well

The behavior of LBS is governed by two properties of the (seed, dataset) pair. First, *seed redundancy*: the farther the seed is from a minimal counterfactual, the more redundant edits LBS can prune, and the greater its advantage over heuristics that do not prune. Second, the *locality of the discriminative structure*: when the predicted class depends on a small, localized motif, a minimal counterfactual is reachable through a nearly monotone sequence of small deletions, and LBS converges fast and deeply; when, instead, the class depends on a global property distributed over a large, dense graph, no small deletion preserves validity, the validity landscape is rugged, greedy deletion gets stuck, and LBS must rely on its stochastic escape moves (overshooting and replacement), whose probability of sampling the correct multi-edit move dilutes under a finite oracle budget. In that regime, LBS reaches only a local optimum within its bounded neighborhood and its advantage over a generic local search narrows. These stalls are an explicit target of ongoing work: a learned *selector* that, instead of sampling escape moves blindly, learns from the structure of each instance through its successes and failures and directs the exploration more efficiently.

4.3 Relationship to Adversarial Examples

A graph counterfactual is characterized by three properties that should not be confused: *validity* (a genuine change of the predicted label, which LBS guarantees by construction),

minimality (few edits, which LBS optimizes), and *plausibility* (closeness to the data manifold or to high-density regions). GC-MIN, and therefore LBS, pursues validity and minimality. Plausibility is a distinct and desirable quality, and there is a known tension between minimality and plausibility. We address plausibility here because minimal-edit refinement under a black-box oracle is closely related to adversarial perturbation: both share an objective, since finding a small change in an input that flips the model's output is the optimization underlying both counterfactual explanations and adversarial attacks (Wachter et al., 2018; Freiesleben, 2022; Pawelczyk et al., 2022). Wachter et al. (2018) call a minimal perturbation that crosses the boundary "a counterfactual by another name", Pawelczyk et al. (2022) show that several counterfactual and adversarial methods coincide for particular hyperparameters, and in the graph setting this is analogous to adversarial attacks on GNNs, which seek minimal, often imperceptible, structural perturbations that change a prediction Zügner et al. (2018). The optimization alone, therefore, does not separate the two.

What distinguishes a counterfactual from an adversarial example is not the search procedure, but the status of the resulting instance Browne and Swift (2020). The defining difference is the relationship with the ground-truth label Freiesleben (2022): an adversarial example is, by definition, *misclassified* with respect to the true label (it leads the model to a label that the instance does not actually have), whereas a counterfactual carries no such requirement, and a counterfactual that genuinely belongs to the target class is not adversarial. Plausibility is *not* part of this definition: a valid off-manifold flip is still a counterfactual, only less plausible. It is, instead, a widely recommended *desideratum* Laugel et al. (2019): keeping a counterfactual close to real data makes it more realistic and reduces the risk that the label flip is a model artifact in a region without support (that is, adversarial in its effects).

In our decoupled formulation, these concerns are clearly separated. LBS produces valid counterfactuals and minimizes their edits. It does not optimize plausibility, and we do not claim that its outputs are on-manifold. By construction, the seed and every solution that LBS accepts are valid counterfactuals, but they need not be real instances of the dataset: Whether the seed is real depends on the generator (DCE and DCEM return real, in-distribution instances of the target class, while the other generators synthesize graphs that need not be real). What keeps the produced explanations on the counterfactual side and not on the adversarial side is fixed, therefore, at the *generation* stage: starting from a real, correctly classified instance of the target class anchors the search to a genuine class change rather than to a perturbation that fools the model, and any domain requirement (for example, the chemical validity of molecules) can be incorporated into the black-box oracle as an additional validity check that every edit, and therefore every LBS solution, must satisfy. Minimization thus remains agnostic to the generator, while plausibility and non-adversariality are couplable concerns of generation and of constraints. Empirically, LBS also minimizes much more effectively starting from real, in-distribution seeds (Section 5.2), although this reflects the quality of the minimization, not a guarantee of plausibility.

5 Experimental Analysis

We evaluate LBS on GC-MIN to test whether explicit minimization produces counterfactuals with fewer structural/feature edits, remains computationally practical, and generalizes across generators and datasets. Experiments on synthetic and real-world graphs (molecular/

biomedical) combine LBS with multiple generators and compare to existing minimization methods; Sections 5.1, 5.2, 5.3 detail the setup, results, and ablations.

5.1 Experimental Setting

5.1.1 Datasets

We consider two synthetic datasets, *Tree-Cycles (TCR)* Ying et al. (2019) and *Synthie* Morris et al. (2016), two real-world biomedical/molecular datasets, *Autism Spectrum Disorder (ASD)* Abrate and Bonchi (2021) and *Brain Blood Barrier Penetration (BBBP)* Wellawatte et al. (2022), and five TUDataset Morris et al. (2020) benchmarks: *ENZYMES*, *BZR*, *PROTEINS*, *AIDS*, and *COLORS-3*. *TCR* consists of tree-structured graphs augmented with cycle motifs, binary-classified by cycle presence. *Synthie* is a multi-class synthetic benchmark with node attributes. *ASD* contains fMRI brain networks from 49 children with Autism Spectrum Disorder and 52 controls; nodes represent brain regions and edges encode functional co-activation. *BBBP* is a molecular dataset for predicting blood–brain barrier permeation, with atom-level node attributes. *ENZYMES* and *PROTEINS* are protein graph benchmarks where labels denote functional classes. *BZR* and *AIDS* are small-molecule cheminformatics datasets with biological activity labels. *COLORS-3* is a synthetic graph-coloring benchmark. These datasets span synthetic motifs, molecular graphs, and brain networks, covering diverse structural and attribute configurations (Table 2).

5.1.2 Evaluation Metrics

We adopt three metrics from Prado-Romero et al. (2024). *Graph Edit Distance (GED)* counts structural modifications (edge additions/removals) between the original graph and its counterfactual; lower is better. *Correctness* is the fraction of instances for which $\Phi(G'') \neq \Phi(G)$. *Oracle Calls* counts queries to the black-box model, serving as a proxy for computational cost. In a model-agnostic setting, oracle evaluation dominates wall-clock time, as each call requires a full forward pass through an arbitrary black-box model, whereas LBS's algorithmic overhead is negligible by comparison. We therefore report oracle calls rather than wall-clock time, as the latter would conflate minimizer efficiency with the cost of the specific oracle used. We additionally introduce *Feature Edit Distance (FED)*, the average absolute difference between corresponding node attribute values, capturing attribute-level

Table 2 Statistical description of the datasets used to evaluate our proposal

Dataset	# of instances	Avg # of nodes	Avg # of edges	# attr	# classes	Graph type	CCs
Tree-Cycles	500	28	27.566	–	2	Undirected	One
Synthie	400	95	172.93	15	4	Undirected	Multiple
ASD	101	116	655.624	–	2	Undirected	Multiple
BBBP	2039	24.06	25.95	9	2	Undirected	One
ENZYMES	600	32.63	62.14	18	6	Undirected	One
BZR	405	35.75	38.36	3	2	Undirected	One
PROTEINS	1113	39.06	72.82	1	2	Undirected	One
AIDS	2000	15.69	16.20	4	2	Undirected	One
COLORS-3	10500	61.31	91.03	4	11	Undirected	One

minimality. Since GC-MIN targets minimality while preserving validity, GED and FED are the primary quality metrics.

5.1.3 Generators and Baselines

We pair LBS with three generators spanning different paradigms: *DCE* Faber et al. (2020) (search-based), *OFS* Abrate and Bonchi (2021) (heuristic-based), and *RSGG* Prado-Romero et al. (2024) (generative). We also include *DCEM*, a medoid-based variant of *DCE* that selects one medoid per class during training, ensuring generator-side oracle complexity independent of dataset size. As baseline minimizer, we use the *obs* strategy from *Oblivious Backward Search* Abrate and Bonchi (2021). Although *OFS* and *obs* are originally coupled, our evaluation decouples generation from minimization for systematic comparison. *obs* is the only existing GCE minimization strategy that can be decoupled from its generation phase, and one that, paired with *OFS*, consistently produces counterfactuals with low GED Prado-Romero et al. (2024), making it the strongest available standalone baseline. To strengthen the comparison, we consider two additional baselines. *DBS* Abrate and Bonchi (2021) is the data-driven backward-search minimizer (the minimization phase of the *DDBS* method, used here decoupled from its generation). *RHC* is a generic hill-climbing minimizer restricted to the same edit operations as LBS (deletion, replacement, addition) but without its prioritized strategy ordering or its overshooting, executed under a matched oracle-call budget. It isolates whether the advantage of LBS comes from its search strategy and not from additional access to the oracle. A natural question is whether general-purpose metaheuristics (e.g., simulated annealing, tabu search) could serve as additional baselines. However, naïvely applying such methods to GC-MIN is non-trivial: the feasibility constraint ($\Phi(G'') \neq \Phi(G)$) is defined by a black-box oracle providing no gradient or distance-to-boundary information, and every candidate evaluation requires an oracle call. Adapting these methods would require designing domain-specific move operators and acceptance criteria, effectively producing new GCE minimizers, which we consider a valuable future direction rather than a baseline comparison. All experiments and code use the *GRETEL*¹ framework (Prado-Romero & Stilo, 2022; Prado-Romero et al., 2023).

5.1.4 Reporting and Reproducibility

All reported metrics are averages over the full test set of each dataset (101 to 10,500 graphs). The large sample sizes, particularly for BBBP (2,039), AIDS (2,000), and COL-ORS-3 (10,500), yield stable estimates, and the observed effect sizes (GED reductions of one to two orders of magnitude, e.g., from 189.71 to 2.17 on Synthie) make improvements robust to instance-level variability. Under our decoupled protocol, each generator is run once per dataset, and all minimizers operate on the same set of valid initial counterfactuals. Correctness is therefore a property of the generator and identical across minimizers, so that the only stochasticity whose stability must be evaluated is that of LBS. We report it directly in a dedicated stability table (Table 4): LBS paired with *DCE* over the four datasets of the figure, across three random seeds, as mean $\pm$ standard deviation. All experimental configurations are publicly released for replication.

¹ Source code and all experimental configurations are publicly available at <https://github.com/aiim-research/GRETEL>.

5.2 Results

We structure our empirical analysis around three research questions:

- **RQ1:** Does explicitly optimizing GC-MIN via LBS improve counterfactual minimality compared to heuristic backward refinement?
- **RQ2:** What is the computational trade-off associated with bounded local search?
- **RQ3:** How does the choice of generator influence refinement quality and scalability?

We present RQ1 and RQ2 as our *main* results, grounded in the *within-generator* comparison (LBS against the baselines applied to the same initial counterfactuals). RQ3 is a *secondary* cross-generator and scalability analysis, interpreted with caution. Each question opens with its motivation and closes with a one-sentence takeaway.

5.2.1 Main Results: Within-Generator Minimality and Cost

RQ1: does explicit minimization improve minimality? *Motivation.* LBS is only worthwhile if explicitly solving GC-MIN produces more minimal counterfactuals than the backward heuristic refinement used in practice. We address RQ1 and RQ2 by comparing LBS against *obs* (and the additional baselines DBS and RHC) across three generators (DCE, OFS, RSGG) and four datasets (ASD, TCR, BBBP, Synthie); results appear in Figures 2, 3, 4. Across all generators and datasets, LBS achieves substantially lower GED than both the standalone generator and *obs*, with particularly large margins for DCE and RSGG. A similar trend holds for FED on attributed datasets, confirming that LBS refines both structural

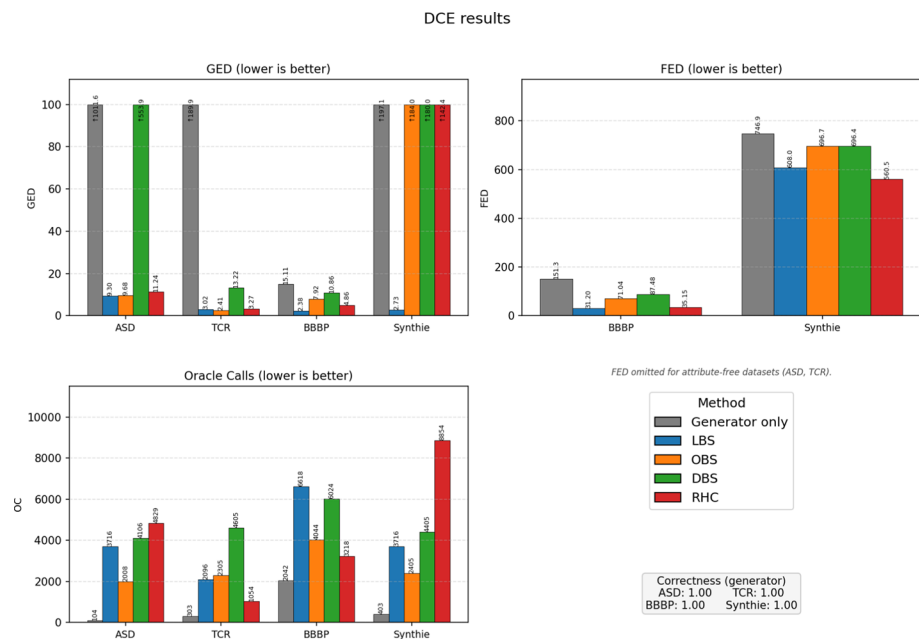


Fig. 2 Results for the DCE generator. The shared chart layout is described in the text (Section 5.2.1)

OFS results

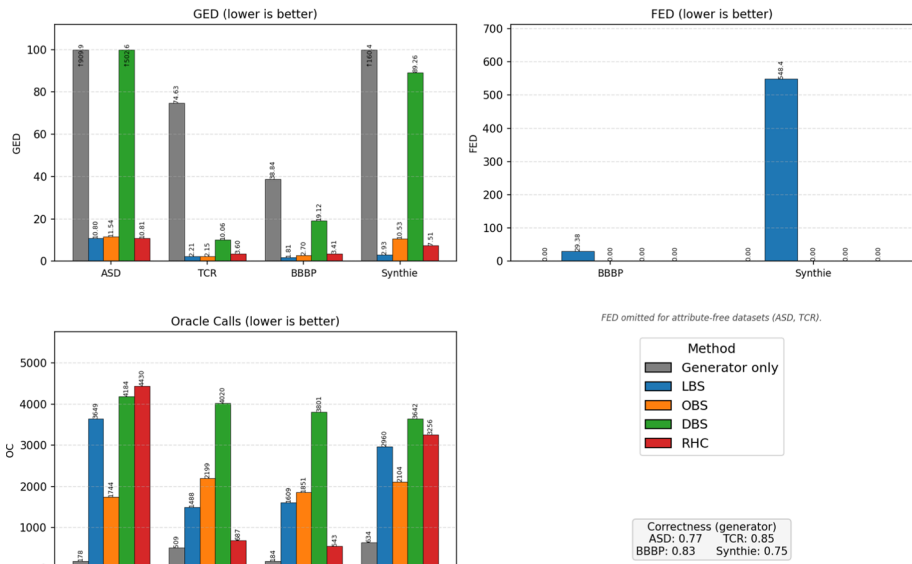


Fig. 3 Results for the **OFS** generator. The shared chart layout is described in the text (Section 5.2.1)

RSGG results

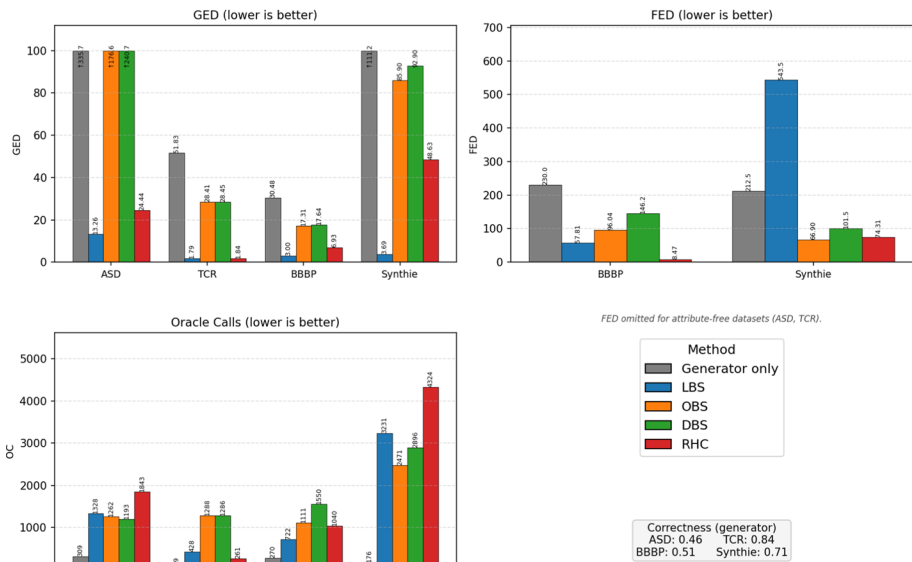


Fig. 4 Results for the **RSGG** generator. The shared chart layout is described in the text (Section 5.2.1)

and attribute-level perturbations. For certain generators (e.g., RSGG on Synthie), LBS may increase FED while reducing GED, reflecting a structure–attribute trade-off. Figures 2, 3, 4 share the same layout: grouped bar charts of GED, FED, and Oracle Calls under a single shared legend, where each group compares the standalone generator against LBS, OBS, and the DBS and RHC baselines applied to the same initial counterfactuals. FED is shown only for the attributed datasets (BBBP, Synthie), as it is not meaningful on the attribute-free ones. In the GED panel, the axis is capped at 100, so longer bars run off the top, and the exact value is printed on top of every bar. Correctness is a property of the generator and is reported once (Table 3), not as a per-minimizer metric.

LBS achieves more modest improvements on certain datasets. On AIDS, the GED reduction is limited (−3.19), likely due to the small average graph size (15.69 nodes), which restricts removable edits. On PROTEINS, residual GED remains high (45.86), suggesting that many edits are individually necessary for validity. These cases indicate that the effectiveness of bounded local search depends on the structural redundancy in the initial counterfactual. Overall, these findings positively answer RQ1: explicitly solving GC-MIN yields substantially more minimal counterfactuals.

5.2.1.1 Correctness and the Evaluation Set Correctness remains unchanged across minimizers, as both LBS and *obs* preserve $\Phi(G'') \neq \Phi(G)$. Since the minimizers operate on the same valid counterfactuals, correctness is a property of the generator, not a discriminating quantity per minimizer. We therefore report it once per generator rather than alongside GED/FED. Its absolute value varies across generators: DCE achieves full correctness, whereas OFS and RSGG exhibit lower rates on certain datasets (e.g., RSGG on BBBP: 0.51). Since GED and FED are computed only over valid counterfactuals, the effective evaluation set differs across generators. Thus, GED/FED comparisons are valid *within* a given generator (between LBS and *obs* applied to the same initial counterfactuals), but *cross-generator* comparisons should be interpreted with caution, as they may reflect different subpopulations rather than purely algorithmic differences.

5.2.1.2 Stochastic Stability Since the generator is run once and all minimizers share its counterfactuals, the only stochasticity to evaluate is that of LBS itself. Table 4 reports LBS paired with DCE over the four datasets of the figures across three random seeds (mean $\pm$ standard deviation). The per-seed variance is small relative to the observed effect sizes

Table 3 Correctness per generator across datasets under the decoupled protocol

Dataset	DCE	OFS	RSGG
Tree-Cycles	1.00	0.85	0.84
ASD	1.00	0.77	0.46
BBBP	1.00	0.83	0.51
Synthie	1.00	0.75	0.71

Since all minimizers operate on the same valid counterfactuals, correctness is a property of the generator and is reported once here (LBS runs) rather than as a per-minimizer metric in Figures 2, 3, 4

Table 4 Stochastic stability of LBS across different random seeds under the decoupled protocol (LBS combined with DCE, over the four datasets in the figures)

Dataset	GED	FED	Oracle Calls
Tree-Cycles	2.48 ± 0.37	-	1305.3 ± 528.1
ASD	9.58 ± 0.24	-	3777.5 ± 101.5
Synthie	2.53 ± 0.34	589.64 ± 12.40	2853.5 ± 577.1
BBBP	2.53 ± 0.11	29.51 ± 1.93	4702.2 ± 1277.0

Each cell reports the mean $\pm$ standard deviation over three seeds. "-" indicates a metric that does not apply (datasets without node attributes)

(GED reductions of one to two orders of magnitude), so that the reported improvements are robust to random initialization.

RQ2: what is the computational trade-off? *Motivation.* A broader neighborhood exploration could be prohibitively costly, so we quantify the cost in oracle calls relative to *obs* and check whether the advantage is merely a budget effect. Regarding RQ2, LBS requires more oracle calls than *obs*, reflecting broader neighborhood exploration. The two methods operate under different budget regimes by design: *obs* exhausts a fixed budget with negligible GED improvements (confirmed in RQ4, Table 6), whereas LBS employs convergence-based early termination and typically uses only a fraction of its budget. Crucially, even under the same budget (e.g., 2.5k calls on Synthie), LBS reduces GED to 16.23 vs. 183.36 for *obs*, demonstrating that the advantage stems from the search strategy rather than additional oracle access. On Synthie and COLORS-3, LBS achieves GED reductions exceeding 95% with moderate additional queries, whereas on structurally constrained datasets, such as AIDS, the marginal improvement is smaller. *Takeaway.* The gains of LBS come from its search strategy, not from additional oracle access: under a matched budget, it still substantially outperforms *obs*.

5.2.2 Cross-Generator Analysis and Scalability

RQ3: how does the generator choice affect refinement and scalability? *Motivation.* Under the decoupled protocol, the generator only provides a seed, so we examine how much that choice still matters for the final minimality and at what generator-side cost. For RQ3, DCE paired with LBS achieves the strongest minimality, plausibly because DCE returns in-distribution graphs, whereas generative approaches such as RSGG may produce out-of-distribution graphs. Starting from an in-distribution seed facilitates refinement, as the search remains in realistic regions of graph space, consistent with Prenkaj et al. (2025). However, DCE's oracle calls scale with dataset size (e.g., $> 10^4$ on COLORS-3). To address this, we evaluate DCEM (Section 5.1), whose generator-side oracle complexity is independent of dataset size.

Table 5 shows that DCEM+LBS achieves minimality comparable to DCE+LBS while reducing generator-side oracle calls to a constant (equal to the number of classes). This reinforces a central insight: once minimization is explicitly optimized, the generator primarily provides a valid seed, while the refinement phase determines explanation minimality. *Takeaway.* The choice of generator mainly affects the quality and cost of the seed, not the achievable minimality: a constant-cost generator (DCEM) paired with LBS matches the best variable-cost generator.

Table 5 Results for DCE/DCEM paired with LBS or *obs* across datasets

Dataset	Explainer	GED	GED-diff	FED	FED-diff	OC	OC-diff
Synthie	DCE	189.71		746.55		403.00	
	DCE-LBS	2.17	-187.54	593.47	-153.08	2422.65	+2019.65
	DCE-obs	177.51	-12.20	697.03	-49.53	4406.00	+4003.00
	DCEM	201.41		753.30		3.00	
	DCEM-LBS	2.16	-199.26	593.61	-159.69	2021.89	+2018.89
	DCEM-obs	185.22	-16.19	694.47	-58.83	4006.00	+4003.00
BBBP	DCE	15.11		151.31		2042.00	
	DCE-LBS	2.38	-12.73	55.62	-95.69	8811.51	+6769.51
	DCE-obs	12.08	-3.03	117.77	-33.54	6019.50	+3977.50
	DCEM	18.98		226.29		3.00	
	DCEM-LBS	3.20	-15.78	65.27	-161.01	6957.12	+6954.12
	DCEM-obs	14.43	-4.55	166.52	-59.76	4006.00	+4003.00
ENZYMES	DCE	55.24		1715.53		603.00	
	DCE-LBS	2.60	-52.65	590.13	-1125.40	2181.34	+1578.34
	DCE-obs	15.54	-39.70	409.85	-1305.68	4606.00	+4003.00
	DCEM	58.46		2034.56		3.00	
	DCEM-LBS	2.55	-55.91	602.84	-1431.72	1563.99	+1560.99
	DCEM-obs	14.46	-44.00	369.51	-1665.04	4006.00	+4003.00
BZR	DCE	25.78		68.62		408.00	
	DCE-LBS	1.84	-23.94	30.65	-37.97	2200.46	+1792.46
	DCE-obs	22.35	-3.43	59.95	-8.67	4341.73	+3933.73
	DCEM	38.06		113.03		3.00	
	DCEM-LBS	2.06	-36.00	30.83	-82.20	1734.13	+1731.13
	DCEM-obs	28.79	-9.26	86.87	-26.16	4006.00	+4003.00
AIDS	DCE	15.09		92.82		2003.00	
	DCE-LBS	11.90	-3.19	70.26	-22.56	12309.93	+10306.93
	DCE-obs	14.91	-0.19	91.43	-1.39	6006.00	+4003.00
	DCEM	17.25		91.35		3.00	
	DCEM-LBS	13.02	-4.23	67.26	-24.09	10924.29	+10921.29
	DCEM-obs	16.92	-0.32	89.47	-1.88	4006.00	+4003.00
PROTEINS	DCE	67.45		227.44		1116.00	
	DCE-LBS	45.86	-21.59	159.88	-67.56	14107.63	+12991.63
	DCE-obs	58.95	-8.50	189.52	-37.92	5119.00	+4003.00
	DCEM	73.19		228.92		3.00	
	DCEM-LBS	50.18	-23.01	167.62	-61.29	13460.50	+13457.50
	DCEM-obs	64.58	-8.61	198.75	-30.17	4006.00	+4003.00
COLORS-3	DCE	91.22		61.53		10503.00	
	DCE-LBS	8.97	-82.25	29.16	-32.37	12194.38	+1691.38
	DCE-obs	81.16	-10.07	54.63	-6.90	14369.62	+3866.62
	DCEM	92.96		62.84		3.00	
	DCEM-LBS	9.18	-83.78	29.22	-33.62	1793.80	+1790.80
	DCEM-obs	82.96	-10.00	55.87	-6.97	3990.33	+3987.33

Table 6 Ablation study of maximum oracle call budget on the Synthie dataset

Max OC	GED		FED		Minimizer OC	
	LBS	obs	LBS	obs	LBS	obs
2.5k	16.23	183.36	611.82	694.82	2067.29	2501.00
5k	12.58	183.38	612.40	694.73	2426.12	5000.00
7.5k	9.58	184.01	613.54	696.68	2662.22	7501.00
10k	2.68	184.01	614.45	696.68	3308.48	10000.00
15k	2.51	183.94	603.55	696.68	4068.43	15000.00
20k	2.50	183.91	610.02	696.68	4447.32	20000.00

The budget cap is applied *per instance*. GED, FED, and oracle calls are averages over the instances, with termination by convergence or by budget exhaustion. "Minimizer OC" counts only the calls made by the minimizer. The generator-side calls are constant at 405 on Synthie (the number of instances, one oracle call per instance, a property of the generator), and have been subtracted from the values reported here

Table 7 Effect of enabling attribute manipulation within LBS across datasets and generators

Dataset	Generator	GED		FED		Oracle Calls	
		no attr	attr	no attr	attr	no attr	attr
Synthie	OFS	7.92	2.92	0.00	120.11	2997.33	2959.58
	DCE	7.28	2.73	193.84	608.03	6767.42	3715.97
	RSGG	17.94	3.69	17.88	543.45	3020.30	3231.07
BBBP	OFS	2.38	1.81	0.00	29.38	421.96	1609.09
	DCE	3.49	2.38	19.49	31.20	3084.40	6617.58
	RSGG	5.29	0.69	31.13	57.81	927.76	721.64

5.3 Ablation Study

We further analyze LBS through targeted ablations on four additional research questions: (RQ4) sensitivity to the oracle budget, (RQ5) the impact of attribute manipulation, (RQ6) the role of neighborhood design, and (RQ7) the role of strategy ordering.

5.3.1 RQ4: Sensitivity to Oracle Budget

Table 6 reports the performance of LBS and *obs* on *Synthie*, selected for its combination of node attributes, multi-class structure, and moderate graph size, under increasing oracle call budgets. *obs* exhausts the allowed budget with negligible GED improvements. In contrast, LBS does not saturate the budget and exhibits clear convergence: GED improves substantially between 2.5k and 10k calls, after which returns diminish, indicating that the search stabilizes once a near-optimal local solution is reached. *Takeaway*. LBS converges within its budget rather than merely spending it. Beyond a moderate budget, additional oracle calls yield diminishing returns.

5.3.2 RQ5: Impact of Attribute Manipulation

Table 7 reports the effect of enabling attribute-level perturbations within LBS for *Synthie* and *BBBP*, the two attributed datasets where the attribute–structure interaction is most pronounced (15 and 9 features, respectively). Enabling attribute manipulation significantly improves GED across generators, demonstrating that structural and attribute refinements

interact synergistically: feature adjustments enable LBS to escape configurations that cannot be improved through edge modifications alone. While FED may increase due to active feature perturbations, structural minimality improves, and correctness remains unaffected, confirming that attribute-level neighborhoods are crucial for GC-MIN in attributed domains. *Takeaway.* On attributed datasets, attribute moves are essential, trading a modest increase in FED for large gains in structural minimality.

5.3.3 RQ6: Role of Neighborhood Design

Table 8 compares the full LBS algorithm with two restricted variants: (i) Strategies 1a & 1b only (removal operations), and (ii) Strategies 2 & 3 only (swap and addition). We select four datasets spanning attributed (BBBP, Synthie) vs. attribute-free (TCR, ASD), using DCE to isolate neighborhood design effects from generator variability. The full algorithm is the most robust variant: using only swap/add operations (Var 2) leads to catastrophic degradation on attribute-free graphs (e.g., GED of 95.09 on TCR), and removal-only operations (Var 1) fail on attributed datasets (e.g., GED of 25.70 on Synthie), while the combined strategy avoids such failures. On attribute-free datasets, the removal-only variant achieves slightly lower GED, indicating that swap/add operations are less beneficial without node attributes. Conversely, on attributed datasets, the full strategy substantially outperforms both restricted variants, as the interplay between structural and attribute-level exploration unlocks solutions neither component can reach alone. *Takeaway.* Removal and swap/addition operations are complementary. Only their combination is robust on both attributed and attribute-free graphs.

5.3.4 RQ7: Role of Strategy Ordering

Beyond *which* strategies are included (RQ6), the priority *order* in which they are attempted under first-improvement acceptance also shapes the search. Table 9 compares the default order against two reorderings, on four datasets with attributes (BBBP, Synthie) and without them (TCR, ASD), using DCE to isolate the effect of the order from generator quality. Var. 3 defers removal entirely, attempting compensating additions and then replacements before any pruning (add, swap, 1a, 1b), and Var. 4 attempts overshooting (Strategy 1b) before single-edit removal (Strategy 1a) (add, 1b, 1a, swap). Keeping the set of strategies fixed, this isolates the contribution of the order itself. The default order is the most robust across the four datasets: deferring removal (Var. 3) is catastrophic on attribute-free graphs (GED 81.58 on TCR and 128.82 on ASD, versus 3.02 and 9.30 for the default), since with no attributes to exploit the early repair finds no improvement and the budget is spent without pruning, whereas bringing overshooting forward (Var. 4) stays close to the default and occasionally matches or slightly beats it (GED 2.53 vs. 2.73 on Synthie) but loses on ASD. *Takeaway.* A

Table 8 DCE generator neighborhood exploration variants across datasets LBS

Dataset	GED			FED			Oracle Calls		
	LBS	Var 1	Var 2	LBS	Var 1	Var 2	LBS	Var 1	Var 2
BBBP	2.38	5.51	3.67	30.89	43.73	40.89	6617.58	4793.26	5755.75
Synthie	2.93	25.70	2.96	548.36	657.12	597.78	2959.58	3861.27	3332.01
TCR	3.02	2.20	95.09	-	-	-	1488.19	832.59	9580.15
ASD	10.8	10.03	146.02	-	-	-	3649.10	5285.04	10931.56

Table 9 Strategy-ordering ablation of Bounded Local Search (variants 3 and 4) across multiple datasets using the DCE generator

Dataset	GED			FED			Oracle Calls		
	bls	Var 3	Var 4	bls	Var 3	Var 4	bls	Var 3	Var 4
BBBP	2.38	3.00	2.94	31.20	38.74	37.60	6617.58	6048.53	5487.69
Synthie	2.73	4.07	2.53	608.03	578.95	571.41	3715.97	3054.64	3014.95
TCR	3.02	81.58	3.26	-	-	-	2095.82	5765.84	1940.39
ASD	9.30	128.82	17.07	-	-	-	3716.42	7164.41	6227.01

Var 3 uses the order add, swap, 1a, 1b, and Var 4 the order add, 1b, 1a, swap, compared against the canonical order (bls)

different ordering does change the results: checking small removals first, as in the default order, is the most robust choice across attributed and attribute-free graphs.

Overall, the ablation results demonstrate that controlled oracle budgeting, attribute-aware perturbations, complementary neighborhood operators, and a sensible strategy ordering are all essential to effectively approximate GC-MIN.

6 Conclusion

We formalized Graph Counterfactual minimization (GC-MIN), proved its NP-hardness, and introduced a decoupled *generate-and-minimize* framework that separates counterfactual construction from refinement. Within this framework, we proposed Local Bounded Search (LBS), a model-agnostic bounded local search algorithm that refines any valid counterfactual by exploring structured neighborhoods of graph edits, optimizing structural and attribute minimality while preserving correctness.

Experiments across nine datasets show that LBS consistently yields more minimal explanations than heuristic refinement, with GED reductions of up to 98%. The introduction of DCEM further addresses generator scalability by ensuring constant oracle complexity, while preserving minimality when paired with LBS. Ablation studies confirm that bounded exploration, attribute-aware perturbations, and complementary neighborhood operators are all essential to the method's effectiveness.

A central implication is that GCE system design can shift emphasis toward principled, generator-agnostic refinement: even a simple valid seed, when paired with structured minimization, yields highly minimal explanations. Future directions include adaptive strategy selection, comparison with alternative metaheuristic minimizers (e.g., simulated annealing, tabu search), integration of domain-specific plausibility constraints, and extension to richer edit spaces.

Appendix A Contribution Summary

This paper formalizes the Graph Counterfactual Explanation problem and proves that its minimization variant is NP-hard. We formally introduce a decoupled generate-and-minimize framework and propose a bounded local search heuristic that improves minimality in practice across diverse datasets. We further analyze scalability and introduce a medoid-

based generator that reduces oracle complexity. Together, these contributions establish an optimization-driven perspective on graph counterfactual refinement.

What is the Main Claim of the Paper? Why is this an Important Contribution to the Machine Learning Literature?

Main Claim

This paper establishes that the minimization of Graph Counterfactual Explanations (GC-MIN) is an NP-hard combinatorial optimization problem and shows that explicitly formulating counterfactual refinement as an optimization objective leads to more minimal explanations in practice.

We introduce a decoupled *generate-and-minimize* framework and propose a model-agnostic bounded local search (LBS) heuristic that refines valid graph counterfactuals by directly targeting structural and attribute minimality under oracle access.

Importance

Graph counterfactual explanations are increasingly used to interpret graph-based machine learning models, yet most existing methods rely on implicit or loosely defined refinement heuristics. By formally characterizing the complexity of graph counterfactual validity and minimization, and by introducing a structured generate-and-minimize formulation, this work:

- Clarifies the theoretical difficulty of achieving minimality;
- Provides an explicit optimization perspective on counterfactual refinement;
- Enables systematic comparison of refinement strategies independently of generation;
- Demonstrates that optimization-driven heuristics can achieve stronger minimality in practice. Rather than claiming exact optimization, we show that grounding refinement in an explicit objective yields measurable empirical improvements across domains.

What is the Evidence You Provide to Support Your Claim? Be Precise

The claim is supported by both theoretical analysis and empirical results.

Theoretical Evidence

- We prove that the decision version of GC-MIN (whether a valid counterfactual within a given edit budget exists) is NP-complete, and that the corresponding optimization problem is NP-hard. These results explain why exact minimality is computationally intractable and justify the use of approximation heuristics.

Empirical Evidence

We evaluate LBS within a decoupled generate-and-minimize framework across multiple synthetic and real-world datasets from molecular and biomedical domains, using several counterfactual generators (DCE, OFS, and RSGG). The main findings are:

- **Improved minimality in practice:** LBS consistently reduces Graph Edit Distance (GED) and Feature Edit Distance (FED) compared to heuristic backward refinement (*obs*), while preserving correctness.
- **Quality–efficiency trade-off:** Although LBS increases oracle usage relative to simpler heuristics, the reduction in structural and attribute perturbations yields a favorable empirical trade-off.
- **Generator interaction:** Refinement performance depends on the initial counterfactual seed. In-distribution generators (e.g., DCE) provide stronger seeds for local search than fully generative approaches.
- **Scalability analysis:** We identify a scalability limitation in DCE, whose oracle complexity scales with dataset size, and introduce DCEM, a medoid-based generator with constant generator-side oracle complexity. When combined with LBS, DCEM preserves minimality while significantly improving scalability.
- **Ablation validation:** Controlled experiments show that bounded exploration, attribute-aware perturbations, and complementary neighborhood operators are all necessary for effective approximation of the GC-MIN objective. Together, these results demonstrate that an explicit optimization-driven refinement heuristic yields consistently more minimal counterfactual explanations in practice.

What Papers by Other Authors Make the Most Closely Related Contributions, and How Is Your Paper Related to Them?

The most closely related works include:

- Abrate and Bonchi (2021), which generate graph counterfactuals and apply heuristic refinement strategies.
- Prenkaj et al. (2025), which studies graph counterfactual generation and reduction using graph style transfer. Also, highlights the role of seed quality in refinement. These works focus primarily on generating valid counterfactual graphs and typically rely on greedy or heuristic refinement procedures. They do not formally characterize the computational complexity of graph counterfactual validity and minimization, nor do they explicitly decouple generation from refinement to study minimization independently.

In contrast, this paper:

- Provides a formal complexity analysis of GC and GC-MIN;
- Introduces a decoupled generate-and-minimize framework;
- Proposes a model-agnostic refinement heuristic grounded in an explicit minimality objective;
- Systematically analyzes generator–minimizer interactions;

- Introduces a scalable generator (DCEM) addressing oracle-complexity limitations.

Have You Published Parts of Your Paper Before, for Instance in a Conference? If So, Give Details of Your Previous Paper(s) and a Precise Statement Detailing How Your Paper Provides a Significant Contribution Beyond the Previous Paper(s)

No part of this research has been previously published. While the authors have published prior work related to graph counterfactual generation, surveys of the field, and evaluation frameworks, the theoretical complexity analysis of GC and GC-MIN, the generate-and-minimize formulation, the LBS heuristic, the DCEM generator, and the comprehensive empirical evaluation presented in this manuscript are entirely new contributions.

Author Contributions Rodrigo García contributed to: Conceptualization, Data curation, Formal analysis, Methodology, Investigation, Software, Writing – original draft. Mario Alfonso Prado-Romero contributed to: Conceptualization, Methodology, Investigation, Supervision, Software, Writing – original draft, Writing – review & editing. Francesco Gullo contributed to: Conceptualization, Methodology, Formal analysis, Writing – original draft, Writing – review & editing. Giovanni Stilo contributed to: Conceptualization, Funding acquisition, Methodology, Validation, Supervision, Writing – original draft, Writing – review & editing

Funding Open access funding provided by Luiss University within the CRUI-CARE Agreement.

Data Availability No datasets were generated or analysed during the current study.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Abrate, C., & Bonchi, F. (2021). Counterfactual graphs for explainable classification of brain networks. *Proc. of ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD)* (pp. 2495–2504).
- Aggarwal, C. C., & Wang, H. (Eds.). (2010). *Managing and mining graph data* (p. 600). Springer.
- Belkin, M., & Niyogi, P. (2003). Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6), 1373–1396.
- Bodria, F., Giannotti, F., Guidotti, R., Naretto, F., Pedreschi, D., & Rinzivillo, S. (2023). Benchmarking and survey of explanation methods for black box models. *Data Mining and Knowledge Discovery (DAMI)*, 37(5), 1719–1778.
- Browne, K., & Swift, B. (2020). Semantics and explanation: why counterfactual explanations produce adversarial examples in deep neural networks [arXiv:2012.10076](https://arxiv.org/abs/2012.10076) arXiv preprint.
- Coifman, R. R., & Lafon, S. (2006). Diffusion maps. *Applied and Computational Harmonic Analysis*, 21(1), 5–30.
- Dimitriou, A., Chaidos, N., Lymperaioi, M., & Stamou, G. (2024) Graph edits for counterfactual explanations: A comparative study. arXiv preprint [arXiv:2401.11609](https://arxiv.org/abs/2401.11609). [arXiv:2401.11609](https://arxiv.org/abs/2401.11609) [cs.LG]
- Faber, L., Moghaddam, A. K., & Wattenhofer, R. (2020). Contrastive graph neural network explanation. In: Proc. of the 37th Graph Repr. Learning and Beyond Workshop at ICML 2020, p. 28. Int. Conf. on Machine Learning.
- Freiesleben, T. (2022). The intriguing relation between counterfactual explanations and adversarial examples. *Minds and Machines*, 32(1), 77–109.

- Frieze, A. M., Gionis, A., & Tsourakakis, C. E. (2013). Algorithmic techniques for modeling and mining large graphs (AMaZING). In: Proc. of ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD), p. 1523
- Garey, M. R., & Johnson, D. S. (1990). *Computers and Intractability: A Guide to the Theory of NP-Completeness* (p. 338). W. H. Freeman and Company.
- Glover, F. W., Kochenberger, G. A. (eds.) (2010) Handbook of Metaheuristics, 2nd edn. International Series in Operations Research & Management Science, vol. 146. Springer, New York, NY.
- Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., & Pedreschi, D. (2018). A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5), 1–42.
- Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems* (pp. 1025–1035).
- Hoos, H. H., & Stützle, T. (2004). *Stochastic Local Search: Foundations and Applications*. Morgan Kaufmann.
- Jiang, J., Leofante, F., Rago, A., & Toni, F. (2024). Robust counterfactual explanations in machine learning: A survey. In: Proc. of Int. Joint Conf. on Artificial Intelligence (IJCAI), pp. 8086–8094. <https://www.ijcai.org/proceedings/2024/894>
- Karimi, A., Barthe, G., Schölkopf, B., & Valera, I. (2023). A survey of algorithmic recourse: Contrastive explanations and consequential recommendations. *ACM Computing Surveys (CSUR)*, 55(5), 95–19529. <https://doi.org/10.1145/3527848>
- Karp, R. M. (1972). Reducibility among combinatorial problems. In R. E. Miller, J. W. Thatcher, & J. D. Bohlinger (Eds.), *Complexity of computer computations* (pp. 85–103). Springer.
- Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks [arXiv:1609.02907](https://arxiv.org/abs/1609.02907) arXiv preprint.
- Kondor, R., & Borgwardt, K. M. (2008). The skew spectrum of graphs. *Proceedings of the 25th International Conference on Machine Learning* (pp. 496–503).
- Laugel, T., Lesot, M.-J., Marsala, C., Renard, X., & Detyniecki, M. (2019). The dangers of post-hoc interpretability: Unjustified counterfactual explanations. *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)* (pp. 2801–2807).
- Liu, Y., Chen, C., Liu, Y., Zhang, X., & Xie, S. (2021). Multi-objective explanations of gnn predictions. In: 2021 IEEE International Conference on Data Mining (ICDM), pp. 409–418. IEEE.
- Ma, J., Guo, R., Mishra, S., Zhang, A., & Li, J. (2022). CLEAR: Generative counterfactual explanations on graphs. In A. H. Oh, A. Agarwal, D. Belgrave, & K. Cho (Eds.), *Advances in Neural Information Processing Systems*.
- Ma, Y., & Tang, J. (2021). *Deep Learning on Graphs*. Cambridge University Press. <https://doi.org/10.1017/9781108924184>
- Michiels, W., Aarts, E. H. L., Lenstra, J. K., Velde, J. A. C. M., & Korst, A. E. E. F. (2007). *Theoretical Aspects of Local Search*. Springer.
- Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, 267, 1–38. <https://doi.org/10.1016/J.ARTINT.2018.07.007>
- Morris, C., Kriege, N. M., Bause, F., Kersting, K., Mutzel, P., & Neumann, M. (2020). Tudataset: A collection of benchmark datasets for learning with graphs. In: ICML 2020 workshop on graph representation learning and beyond (GRL+ 2020).
- Morris, C., Kriege, N. M., Kersting, K., & Mutzel, P. (2016). Faster kernels for graphs with continuous attributes via hashing. *IEEE 16th International Conference on Data Mining (ICDM)* (pp. 1095–1100). IEEE.
- Pawelczyk, M., Agarwal, C., Joshi, S., Upadhyay, S., & Lakkaraju, H. (2022). Exploring counterfactual explanations through the lens of adversarial examples: A theoretical and empirical analysis. *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS)*. PMLR (Vol. 151).
- Petch, J., Di, S., & Nelson, W. (2021). Opening the black box: The promise and limitations of explainable machine learning in cardiology. *Canadian Journal of Cardiology*. <https://doi.org/10.1016/j.cjca.2021.09.004>
- Prado-Romero, M. A., & Stilo, G. (2022). Gretel: Graph counterfactual explanation evaluation framework. In: *Proceedings of the 31st ACM International conference on information & knowledge management. CIKM '22*, pp. 4389–4393. Association for Computing Machinery, New York, NY, USA.
- Prado-Romero, M. A., Prenkaj, B., & Stilo, G. (2023). Developing and evaluating graph counterfactual explanation with gretel. *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining* (pp. 1180–1183).
- Prado-Romero, M. A., Prenkaj, B., & Stilo, G. (2024). Robust stochastic graph generator for counterfactual explanations. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(19), 21518–21526. <https://doi.org/10.1609/aaai.v38i19.30149>

- Prado-Romero, M. A., Prenkaj, B., Stilo, G., & Giannotti, F. (2024). A survey on graph counterfactual explanations: Definitions, methods, evaluation, and research challenges. *ACM Computing Surveys*. <https://doi.org/10.1145/3618105>
- Prenkaj, B., Zaradoukas, E., & Kasneci, G. (2025). Graph inverse style transfer for counterfactual explainability. *Proc. of Int. Conf. on Machine Learning (ICML)*.
- Qiu, D., Wang, M., Khan, A., & Wu, Y. (2024). Generating robust counterfactual witnesses for graph neural networks. *arXiv preprint arXiv:2404.19519*. [arXiv:2404.19519](https://arxiv.org/abs/2404.19519) [cs.LG]
- Sandryhaila, A., & Moura, J. M. (2013). Discrete signal processing on graphs. *IEEE Transactions on Signal Processing*, 61(7), 1644–1656.
- Shuman, D. I., Narang, S. K., Frossard, P., Ortega, A., & Vandergheynst, P. (2013). The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. *IEEE Signal Processing Magazine*. <https://doi.org/10.1109/msp.2012.2235192>
- Tan, J., Geng, S., Fu, Z., Ge, Y., Xu, S., Li, Y., & Zhang, Y. (2022). Learning and evaluating graph neural network explanations based on counterfactual and factual reasoning. *Proc. of the ACM Web Conf. 2022* (pp. 1018–1027).
- Verma, S., Armgaan, B., Medya, S., & Ranu, S. (2023). Empowering counterfactual reasoning over graph neural networks through inductivity. *arXiv preprint arXiv:2306.04835*. [arXiv:2306.04835](https://arxiv.org/abs/2306.04835) [cs.LG]
- Wachter, S., Mittelstadt, B., & Russell, C. (2018). Counterfactual explanations without opening the black box: Automated decisions and the GDPR. *Harvard Journal of Law and Technology*, 31(2), 841–887.
- Wellawatte, G. P., Seshadri, A., & White, A. D. (2022). Model agnostic generation of counterfactual explanations for molecules. *Chemical science*, 13(13), 3697–3705.
- Wu, Z., Ramsundar, B., Feinberg, E. N., Gomes, J., Geniesse, C., Pappu, A. S., Leswing, K., & Pande, V. (2018). MoleculeNet: A benchmark for molecular machine learning. *Chemical Science*, 9(2), 513–530.
- Ying, Z., Bourgeois, D., You, J., Zitnik, M., & Leskovec, J. (2019). Gnnexplainer: Generating explanations for graph neural networks. *Advances in neural information processing systems*, 32,
- Zhang, Z., & Zhao, L. (2021). Representation learning on spatial networks. *Proc. of Conf. on Advances in Neural Information Processing Systems (NeurIPS)* (pp. 2303–2318).
- Zhou, D., Bousquet, O., Lal, T. N., Weston, J., & Schölkopf, B. (2004). Learning with local and global consistency. *Advances in Neural Information Processing Systems* (pp. 321–328).
- Zügner, D., Akbarnejad, A., & Günnemann, S. (2018). Adversarial attacks on neural networks for graph data. In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pp. 2847–2856.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Rodrigo García¹ · Mario Alfonso Prado-Romero² · Francesco Gullo³ · Giovanni Stilo⁴

✉ Giovanni Stilo
gstilo@luiss.it

Rodrigo García
rodrigo.garcia21111@gmail.com

Mario Alfonso Prado-Romero
marioalfonso.prado@gransassotech.org

Francesco Gullo
francesco.gullo@univaq.it

¹ University of Havana, Havana, Cuba

² Fondazione Gran Sasso Tech, L'Aquila, Italy

³ University of L'Aquila, L'Aquila, Italy

⁴ Luiss Business School & Università Luiss Guido Carli, Rome, Italy